DATA SCIENCE PROJECT PAC

PROJECT 1: RECOMMENDATION SYSTEM

Title: Smart Product Recommendation System using Collaborative Filtering

Objective

To recommend products/movies based on user behavior using collaborative filtering.

Tech Stack

- Python
- Pandas, NumPy
- Scikit-learn
- Surprise Library

📁 **Folder Structure**

```
Recommendation_System/
├── data/
│   └── ratings.csv
├── model/
├── app.py
├── recommender.py
├── README.md
```

🔄 **Steps**

1. Load dataset (user, item, rating)
2. Create user-item matrix
3. Apply collaborative filtering
4. Train model (KNN / SVD)
5. Generate recommendations

Code

```
import pandas as pd
from surprise import Dataset, Reader, SVD
from surprise.model_selection import train_test_split

# Load data
df = pd.read_csv("ratings.csv")

reader = Reader(rating_scale=(1, 5))
data = Dataset.load_from_df(df[['userId', 'itemId', 'rating']], reader)

trainset, testset = train_test_split(data, test_size=0.2)

# Model
model = SVD()
model.fit(trainset)

# Prediction
pred = model.predict(1, 101)
print(pred)
```

Smart Product Recommendation System using Collaborative Filtering

1. Introduction

In today's digital world, users are exposed to a huge number of products, movies, and services. It becomes difficult for them to find what they actually like. This is where **Recommendation Systems** play an important role.

A recommendation system is a type of data science application that suggests relevant items to users based on their preferences, behavior, or past interactions.

We see recommendation systems everywhere:

- Movie suggestions on streaming platforms
- Product suggestions on e-commerce websites
- Music recommendations on apps

This project focuses on building a **Smart Product Recommendation System** using **Collaborative Filtering**, which suggests items based on user behavior.

2. Objective

The main objective of this project is:

- To build a system that recommends products or movies to users
- To use **user-item interaction data (ratings)**
- To apply **Collaborative Filtering techniques**
- To generate **personalized recommendations**

3. Technologies Used

The following tools and technologies are used:

- **Python** – Programming language
- **Pandas** – Data manipulation
- **NumPy** – Numerical operations
- **Scikit-learn** – Machine learning utilities
- **Surprise Library** – Specialized library for recommendation systems

📁 4. Project Structure

Recommendation_System/

├── data/

│ └── ratings.csv

├── model/

├── app.py

├── recommender.py

├── README.md

Explanation:

- **data/** → Contains dataset (user ratings)
- **model/** → Stores trained models
- **app.py** → Main application file
- **recommender.py** → Core recommendation logic
- **README.md** → Project overview

5. Workflow (Step-by-Step)

Step 1: Data Collection

We use a dataset (ratings.csv) containing:

- User ID
- Item ID
- Rating

Example:

userId | itemId | rating

1 | 101 | 4

2 | 102 | 5

Step 2: Data Loading

We load the dataset using Pandas:

df = pd.read_csv("ratings.csv")

Step 3: Data Preparation

We prepare the data in a format suitable for the Surprise library:

reader = Reader(rating_scale=(1, 5))

data = Dataset.load_from_df(df[['userId', 'itemId', 'rating']], reader)

Step 4: Train-Test Split

We split the dataset into training and testing sets:

trainset, testset = train_test_split(data, test_size=0.2)

Step 5: Model Training

We use the **SVD (Singular Value Decomposition)** algorithm:

model = SVD()

model.fit(trainset)

SVD works by:

- Finding hidden patterns in user behavior
- Reducing data into latent factors
- Predicting unknown ratings

Step 6: Making Predictions

We predict how a user would rate an item:

pred = model.predict(1, 101)

print(pred)

This gives:

- Estimated rating

- Error details
- Prediction confidence

Step 7: Generating Recommendations

Based on predicted ratings:

- Items with **higher predicted scores** are recommended
- Each user gets **personalized suggestions**

6. Core Code

```
import pandas as pd
from surprise import Dataset, Reader, SVD
from surprise.model_selection import train_test_split

# Load data
df = pd.read_csv("ratings.csv")

reader = Reader(rating_scale=(1, 5))
data = Dataset.load_from_df(df[['userId', 'itemId', 'rating']], reader)

trainset, testset = train_test_split(data, test_size=0.2)

# Model
model = SVD()
model.fit(trainset)

# Prediction
pred = model.predict(1, 101)
print(pred)
```

7. Output

The system provides:

- Personalized recommendations for each user
- Predicted ratings for unseen items
- Better user experience

Example Output:

User 1 → Recommended Items: 105, 110, 120

Predicted Rating for Item 101 → 4.3

8. Types of Recommendation Systems

1. Content-Based Filtering

- Based on item features
- Example: recommending similar movies

2. Collaborative Filtering (Used in this project)

- Based on user behavior
- Finds similar users or patterns

9. Real-World Use Cases

Recommendation systems are widely used in:

- Movie platforms → Suggest movies based on watch history
- E-commerce → Recommend products
- Music apps → Suggest songs/playlists
- Social media → Suggest friends or content

10. Model Evaluation

The performance can be measured using:

- **RMSE (Root Mean Square Error)**
- **MAE (Mean Absolute Error)**

Lower error means better predictions.

11. Challenges

- Cold Start Problem (new users/items)
- Sparse data (few ratings)
- Scalability issues

12. Conclusion

This project demonstrates how machine learning can be used to build an intelligent recommendation system.

Key learnings:

- Understanding user behavior is powerful
- Collaborative filtering is effective
- Data plays a crucial role in personalization

The system can be further improved by:

- Adding deep learning models
- Using hybrid recommendation systems
- Integrating real-time data

13. Future Enhancements

- Web app integration
- Real-time recommendations
- Hybrid models (Content + Collaborative)
- Deployment using cloud platforms